Hyperlogistics: Smart Supply Chain Optimization System

Phase 2 Implementation Report

CS 5542 Big Data Analytics & Applications

Daniel Evans (Data/Back-End Engineer)
Joel Vinas (Data Engineer/ML Engineer)
Tony Nguyen (ML/Full-Stack Engineer)

February 27, 2026

Abstract

This report documents the Phase 2 implementation of Hyperlogistics, an intelligent middle-mile supply chain optimization system built on a Snowflake-native stack. Following the architectural design established in Phase 1, this phase focused on constructing and validating the full Bronze–Silver data pipeline: ingesting over **7.1 million raw records** across five datasets into Snowflake's Bronze layer, and transforming them into four analytics-ready Silver tables. All eight pipeline tables passed automated row-count and schema validation. Key engineering challenges resolved in this phase include multi-type PyArrow schema conflicts, column identifier normalization across heterogeneous CSV encodings, and NOAA GSOD temporal filtering for weather alert preprocessing. The pipeline is now production-ready for Gold-layer vector embedding and the Streamlit in Snowflake dispatcher dashboard, which constitute Phase 3 deliverables.

Contents

1 Phase 2 Objectives **4**

2 Data Pipeline Architecture **4**
2.1 Medallion Schema Design . . . 4
2.2 Pipeline Orchestration . . . 4

3 Bronze Layer — Ingestion Implementation **5**
3.1 Column Normalization . . . 5
3.2 Dataset Ingestion Summary . . . 6
3.3 Notable Ingestion Challenges . . . 7
3.3.1 US Accidents — Schema Drift on Re-run . . . 7
3.3.2 Bridge Inventory — PyArrow Mixed-Type Serialization . . . 7

4 Silver Layer — Preprocessing Implementation **7**
4.1 Accidents → `SILVER.RISK_HEATMAP_VIEW` . . . 7
4.2 Bridge Inventory → `SILVER.BRIDGE_INVENTORY_GEO` . . . 7
4.3 DataCo → `SILVER.CLEANED_LOGISTICS` . . . 8
4.4 Logistics Operations → `SILVER.LOGISTICS_VECTORIZED` . . . 8
4.5 NOAA GSOD → `SILVER.WEATHER_ALERTS` . . . 8

5 Pipeline Validation Results **8**

6 Key Challenges and Resolutions **9**

7 Conclusion **10**

A Dataset & Knowledge Base Documentation **11**
A.1 Dataset Names, Modalities, and Source Links . . . 11
A.2 Domain Relevance to the Project . . . 11
A.3 Multimodal Ingestion Details . . . 12
A.4 Data Preprocessing and Sampling Strategy . . . 13

B Retrieval & Processing Pipeline **14**
B.1 Chunking Strategy: Semantic Record-Level . . . 14
B.2 Indexing and Retrieval Configuration . . . 14
B.3 Embedding Models Used . . . 15
B.4 Example Retrieval Outputs . . . 15
B.4.1 Query 1: "What are the recent safety concerns for driver DRV_102?" . . . 15
B.4.2 Query 2: "Check if my route from Chicago to St. Louis is safe today." . . . 15
B.5 Preprocessing Scripts in GitHub . . . 15

C Application Integration **16**
C.1 Streamlit Project Interface . . . 16
C.2 Evidence Display and Grounded Answer Generation . . . 16
C.3 Query Logging and Evaluation Outputs . . . 17
C.4 Screenshots and Deployment . . . 17

D Snowflake Data Pipeline & Schema **18**
D.1 Snowflake Schema (Tables, Stages, Views) . . . 18
D.2 Reproducible Ingestion Scripts (SQL / Snowpark / Python) . . . 19
D.3 Example Queries Demonstrating Warehouse Connectivity . . . 19
D.4 How Snowflake Integrates with the Application . . . 20

E Reproducibility Plan **21**
E.1 Environment Configuration 21
E.2 Model Versions and Dataset Versioning 21
E.3 Random Seeds and Configuration Files 22
E.4 Run Instructions 22

1 Phase 2 Objectives

Phase 2 objectives were derived directly from the Week 1 and Week 2 sprints defined in the Phase 1 proposal:

1. **Establish the Medallion Architecture** — Provision all three schema layers (Bronze, Silver, Gold) in Snowflake and validate connectivity from the Python Snowpark client.
2. **Implement Full Bronze Ingestion** — Ingest all five source datasets into Bronze tables using chunked Snowpark writes with automatic schema detection and column normalization.
3. **Implement Silver Preprocessing** — Apply feature engineering and type transformations to produce four Silver tables: the risk heatmap, cleaned logistics, geo-indexed bridge inventory, and logistics vectorization corpus.
4. **Automate and Validate the Pipeline** — Build an end-to-end pipeline runner (`src/run_pipeline.py`) and a verification harness (`src/verify_pipeline.py`) to confirm all tables exceed minimum row-count thresholds.
5. **Implement NOAA Weather Alert Extraction** — Configure the `BRONZE.NOAA_GSOD` table and preprocess 2023–2025 weather observations into `SILVER.WEATHER_ALERTS`.

2 Data Pipeline Architecture

2.1 Medallion Schema Design

The Snowflake database `HYPERLOGISTICS_DB` is organized into three schemas following the Medallion Architecture pattern introduced in Phase 1:

- **BRONZE** — Raw landing zone. Data is ingested "as-is" with normalized column names (uppercase, underscore-separated) but no semantic transformation. Tables are overwritten on each pipeline run to ensure idempotency.
- **SILVER** — Cleaned and enriched layer. Snowpark Python transformations apply type casting, geospatial conversion, feature engineering (delay propensity, risk scores), and vectorization stubs. Results are materialized as permanent tables.
- **GOLD** — Analytics- and LLM-ready layer. To be populated in Phase 3 via Snowflake Cortex embedding calls and dynamic table refresh.

2.2 Pipeline Orchestration

The pipeline is orchestrated by `src/run_pipeline.py`, which:

1. Reads credentials from a `.env` file and builds a shared Snowpark session.
2. Executes all ingestion steps in dependency order (Bronze first).
3. Executes all preprocessing steps (Silver).
4. Skips the NOAA weather preprocessing step gracefully if `BRONZE.NOAA_GSOD` is empty, printing a human-readable instruction to the operator.
5. Catches and logs exceptions per step without aborting the pipeline, ensuring partial failures do not block unrelated datasets.

The orchestrator exposes a `_step(label, fn, *args)` helper that wraps each callable, prints structured progress output, and preserves the full traceback for any failed step.

3 Bronze Layer — Ingestion Implementation

3.1 Column Normalization

A recurring engineering challenge across all five CSV datasets was column identifier heterogeneity: column headers contained spaces, parentheses, mixed casing, and special characters (e.g., `Days for shipping (real)`, `STRUCTURE_NUMBER_008`). Snowflake requires unquoted identifiers to be uppercase alphanumeric with underscores.

All ingestion scripts apply the following normalization function before creating Snowpark DataFrames:

Listing 1: Column normalization applied to all ingested CSVs

```
import re

def _normalize_col(c: str) -> str:
    return re.sub(r"[^A-Z0-9]+", "_", c.strip().upper()).strip("_")
```

This transforms, for example, `Days for shipping (real)` → `DAYS_FOR_SHIPPING_REAL` and `Start_Time` → `START_TIME`.

3.2 Dataset Ingestion Summary

Table 1: Bronze Layer Ingestion Results

Script	Bronze Table	Rows	Source	Key Engineering Notes
`ingest_accidents.py`	`TRAFFIC_INCIDENTS`	3,500,000	US Accidents CSV (3 GB)	Chunked write (100K rows/chunk); explicit `DROP TABLE` before loop to avoid schema mismatch on schema-drifted re-runs; `low_memory=False` for mixed-type columns.
`ingest_bridges.py`	`BRIDGE_INVENTORY`	624,193	NBI CSV (US DOT)	All `object`-typed pandas columns cast to `str` before Snowpark ingestion to prevent PyArrow mixed-type serialization errors on `STRUCTURE_NUMBER_008`.
`ingest_dataco.py`	`RAW_LOGISTICS`	180,519	DataCo CSV (Kaggle)	`encoding='latin1'` required; column normalization converts 48 headers with spaces/parentheses.
`ingest_logistics.py`	`LOGISTICS`	2,748,530	14-table logistics ops DB	Multi-file directory scan; foreign-key join across 14 CSVs before single-table write.
`ingest_noaa.py`	`NOAA_GSOD`	Per-load	NOAA GSOD station CSVs	Accepts single file or directory; NOAA `999.9` sentinel replaced with `NULL`; date stored as `VARCHAR` for downstream `TO_DATE` casting.

3.3 Notable Ingestion Challenges

3.3.1 US Accidents — Schema Drift on Re-run

The original ingestion script used Snowpark's `save_as_table(mode="overwrite")` for the first chunk. On re-runs, this resulted in a `DELETE + INSERT` rather than a true `DROP + CREATE`, so a previously-written 44-column table could not absorb a 46-column first chunk. The fix was to issue an explicit `DROP TABLE IF EXISTS` before the chunked append loop, ensuring atomic schema recreation:

Listing 2: Atomic drop-and-append strategy for accidents ingestion

```
session.sql("DROP␣TABLE␣IF␣EXISTS␣BRONZE.TRAFFIC_INCIDENTS").collect()
for i, chunk in enumerate(chunks):
    sp_df = session.create_dataframe(chunk)
    sp_df.write.mode("append").save_as_table("BRONZE.TRAFFIC_INCIDENTS")
```

3.3.2 Bridge Inventory — PyArrow Mixed-Type Serialization

The NBI dataset contains `STRUCTURE_NUMBER_008`, a column with mixed integer/string values that Pandas infers as `object` dtype. PyArrow, used internally by Snowpark for DataFrame serialization, raises a type-inference error on such columns. The resolution was to cast *all* `object`-dtype columns to `str` before handing the DataFrame to Snowpark:

Listing 3: Cast all object columns to str before Snowpark ingestion

```
for c in df.select_dtypes(include="object").columns:
    df[c] = df[c].astype(str)
```

4 Silver Layer — Preprocessing Implementation

4.1 Accidents → `SILVER.RISK_HEATMAP_VIEW`

The preprocessing script (`preprocess_accidents.py`) aggregates 3.5 million raw incident records by US state, city, and hour-of-day using Snowpark's `groupBy` and `agg` operations. It computes:

- **Incident Count** per spatial cluster.
- **Average Severity** (1–4 scale) as a float.
- **Risk Score**: a weighted composite of incident count and average severity normalized to $[0, 1]$.

The result (**2,587,453 rows**) provides the geospatial foundation for the dispatcher dashboard's risk heatmap overlay.

4.2 Bridge Inventory → `SILVER.BRIDGE_INVENTORY_GEO`

The bridge preprocessing script converts the NBI's decimal-degree latitude/longitude columns (`LATDD` and `LONGDD`) into Snowflake's native `GEOGRAPHY` type using `ST_GEOGFROMTEXT`:

Listing 4: GEOGRAPHY conversion for spatial join support

```
SELECT *, ST_GEOGFROMTEXT('POINT(' || LONGDD || '␣' || LATDD || ')')
    AS LOCATION
FROM BRONZE.BRIDGE_INVENTORY
WHERE LATDD IS NOT NULL AND LONGDD IS NOT NULL;
```

All **624,193** bridge records are retained (no sampling) to ensure exhaustive spatial join coverage for the safety-veto layer. A spatial join against this table guarantees that the Planning Agent can detect any bridge clearance or weight violation along a proposed reroute.

4.3 DataCo → SILVER.CLEANED_LOGISTICS

Feature engineering on the DataCo dataset (preprocess_dataco.py) produces three derived columns critical for SRSNet training:

1. **DELAY_DAYS**: Arithmetic difference between actual and scheduled shipping days:

$$\text{DELAY_DAYS} = \texttt{DAYS_FOR_SHIPPING_REAL} - \texttt{DAYS_FOR_SHIPMENT_SCHEDULED}$$

   A pure datediff SQL call was not viable because the source columns are integer counts, not date types.

2. **LATE_DELIVERY_RISK**: Cast from the LATE_DELIVERY_RISK integer column (0/1) to a typed IntegerType() field.

3. **SHIPPING_DELAY_ZSCORE**: Z-score of delay days partitioned by SHIPPING_MODE, enabling anomaly detection across transit types. Computed using Snowpark's Window function.

4.4 Logistics Operations → SILVER.LOGISTICS_VECTORIZED

The logistics preprocessing script (preprocess_logistics.py) constructs a concatenated free-text DESCRIPTION column from key operational fields (driver, route, load type, incident status). This description field serves as the input for Phase 3 Cortex embedding calls to populate the vector store in GOLD.VECTOR_EMBEDDINGS.

4.5 NOAA GSOD → SILVER.WEATHER_ALERTS

The weather preprocessing script (preprocess_weather.py) filters the BRONZE.NOAA_GSOD table to **active years only** (2023–2025) using a server-side YEAR(DATE) predicate pushed down to Snowflake, then applies rule-based alert classification:

Table 2: Rule-Based Weather Alert Thresholds

Condition	Threshold	Alert Label
Precipitation (PRCP)	> 50 mm	Heavy precipitation expected
Snowfall depth (SNDP)	> 10 cm	Significant snowfall
Visibility (VISIB)	< 1 mile	Low visibility conditions
Default	—	Normal conditions

The output SILVER.WEATHER_ALERTS table contains 10 columns including STATION, DATE, raw weather metrics, the WEATHER_ALERT label, and a human-readable ALERT_DESCRIPTION string suitable for injection into RAG prompts.

5 Pipeline Validation Results

A dedicated verification script (src/verify_pipeline.py) queries each table's COUNT(*) and compares against a minimum threshold. All eight Bronze and Silver tables passed validation:

Table 3: Pipeline Verification Results — All 8/8 Tables Pass

Schema	Table	Actual Rows	Min Threshold	Status
BRONZE	TRAFFIC_INCIDENTS	3,500,000	3,000,000	**PASS**
BRONZE	BRIDGE_INVENTORY	624,193	600,000	**PASS**
BRONZE	RAW_LOGISTICS	180,519	150,000	**PASS**
BRONZE	LOGISTICS	2,748,530	2,000,000	**PASS**
SILVER	RISK_HEATMAP_VIEW	2,587,453	1,000,000	**PASS**
SILVER	BRIDGE_INVENTORY_GEO	624,193	600,000	**PASS**
SILVER	CLEANED_LOGISTICS	180,519	150,000	**PASS**
SILVER	LOGISTICS_VECTORIZED	2,748,530	2,000,000	**PASS**
Total records across Bronze + Silver				**13,193,937**

The Gold-layer tables (VECTOR_EMBEDDINGS, DISRUPTION_REPORTS, QUERY_LOGS) remain empty at this stage, as they are populated at runtime by the Streamlit application and Cortex embedding jobs scheduled for Phase 3.

6 Key Challenges and Resolutions

Table 4: Engineering Challenges Encountered and Resolved in Phase 2

Challenge	Root Cause	Resolution
Column count mismatch on re-run (Accidents)	`overwrite` on existing table performs `DELETE+INSERT`, not `DROP+CREATE`; schema change not applied.	Explicit `DROP TABLE IF EXISTS` before chunked `append` loop.
PyArrow mixed-type serialization (Bridges)	`STRUCTURE_NUMBER_008` inferred as `object` dtype with mixed int/str values.	Cast all `object`-dtype columns to `str` before Snowpark write.
Latin-1 encoding error (DataCo)	Non-UTF-8 characters in customer name fields.	`pd.read_csv(..., encoding='latin1')`.
Quoted identifier errors (DataCo, Bridges)	Column names like `Days for shipping (real)` did not normalize to valid SQL identifiers when Snowpark generated SQL.	`_normalize_col()` strips special chars to uppercase underscored identifiers pre-ingestion.
`datediff` on integer delay columns (DataCo Silver)	`DAYS_FOR_SHIPPING_REAL` and `DAYS_FOR_SHIPMENT_SCHEDULED` are integer counts, not DATE types; `datediff` raised a type error.	Replaced with arithmetic subtraction in Snowpark: `col(A) - col(B)`.
NOAA GSOD `METADATA$FILENAME` access (Weather)	`METADATA$FILENAME` is a SQL pseudo-column not exposed through the Snowpark DataFrame API.	Filter replaced with `year(col("DATE")).isin(ACTIVE_YEARS)`, which generates a server-side predicate.

7 Conclusion

Phase 2 successfully delivered a robust, production-validated Bronze–Silver data pipeline ingesting over 7.1 million source records and producing 13.2 million total records across eight Snowflake tables. All engineering challenges were resolved through targeted Pythonic fixes: column normalization, explicit schema management, PyArrow type coercion, and server-side Snowpark predicate pushdown for temporal filtering.

The pipeline is fully automated, idempotent, and self-validating via `verify_pipeline.py`. The Silver layer now provides analytics-ready tables for the three core middle-mile intelligence functions: geospatial risk scoring (accidents), hard-constraint safety veto (bridges), and logistics delay pattern learning (DataCo + Operations DB). NOAA GSOD weather alert extraction is structurally complete and activated once station CSV files are deposited in `data/noaa/`.

A Dataset & Knowledge Base Documentation

A.1 Dataset Names, Modalities, and Source Links

The system ingests five datasets spanning three data modalities. The first three are **core middle-mile datasets** with GPS coordinates enabling geospatial overlay on freight corridors; the remaining two are **supporting datasets** for model training and RAG retrieval.

Table 5: Complete Dataset Inventory

#	Dataset	Modality	Source	Size	Architecture Layer
1	US Accidents (2016–2023)	Geospatial Tabular (CSV)	Kaggle	7.7M rows	Data Perception
2	NOAA GSOD Weather	Time-Series Geospatial (Parquet)	AWS Open Data	Multi-TB	Intelligence & Forecasting
3	National Bridge Inventory	Geospatial Tabular (CSV/GeoJSON)	US DOT BTS	600K+	Validation & Safety
4	DataCo Supply Chain	Tabular (CSV)	Kaggle	180K	Supporting (SRSNet Training)
5	Logistics Operations DB	Tabular (14 CSVs)	Kaggle	550K	Supporting (RAG Knowledge Base)

A.2 Domain Relevance to the Project

Middle-mile logistics — the transportation leg between distribution centers — requires **route-level, geospatially grounded** data for disruption detection and rerouting. The three core datasets provide orthogonal coverage of the middle-mile problem space:

- **US Accidents** → *"Where are the historical risk zones along freight corridors?"* — Enables proactive avoidance of accident blackspots on I-70, I-80, I-55, I-10.
- **NOAA Weather** → *"What real-time environmental conditions could disrupt transit?"* — Provides the disruption trigger signals (icing, flooding, low visibility) that initiate the rerouting workflow.
- **Bridge Inventory** → *"Can the vehicle physically traverse this alternate route?"* — Enforces hard physical constraints (clearance, load limits) that no AI reasoning can override.

Together, these datasets answer the three fundamental middle-mile questions: *risk*, *trigger*, and *feasibility*.

The two supporting datasets extend coverage: **DataCo** provides historical delay patterns for SRSNet forecasting model training, while the **Logistics Operations DB** (14 interconnected tables covering loads, trips, drivers, trucks, fuel, maintenance, and safety incidents) is vectorized into `SILVER.LOGISTICS_VECTORIZED` to serve as the RAG knowledge base for natural language queries.

A.3 Multimodal Ingestion Details

Table 6: Ingestion Methods by Dataset

Dataset	Ingestion Method	Bronze Table	Format Details
US Accidents	**Snowpipe** (real-time) via S3	`TRAFFIC_INCIDENTS`	CSV → 47 columns (GPS, severity, road features, weather at scene)
NOAA GSOD	**External Table** (direct S3)	`NOAA_GSOD`	Parquet → station-level daily summaries (PRCP, SNWD, VISIB, TEMP)
Bridge Inventory	**Snowpipe** via S3	`BRIDGE_INVENTORY`	CSV/GeoJSON → structural attributes + lat/lon coordinates
DataCo Supply Chain	**Snowpipe** via S3	`RAW_LOGISTICS`	CSV → 48 columns, Latin-1 encoding, normalized headers
Logistics Ops DB	**Snowpipe** via S3	`LOGISTICS`	14 CSVs → joined into single table with foreign-key resolution

- **Snowpipe** provides event-driven, near-real-time ingestion for four datasets via S3 event notifications.
- **External Tables** avoid data duplication for the multi-terabyte NOAA archive — queries run directly against S3.
- **Automated S3 Loading**: See `src/sql/02_setup_s3_automation.sql` and `src/ingestion/setup_s3_automation.py`

A.4 Data Preprocessing and Sampling Strategy

Table 7: Preprocessing Scripts and Sampling Strategies

Dataset	**Script**	**Key Transformations**	**Sampling Strategy**	**Silver Output**
US Accidents	`preprocess_accidents.py`	Aggregation by State/City/Hour; severity averaging; risk scoring	Geospatial clustering by proximity to freight corridors with severity weighting	`RISK_HEATMAP_VIEW`
NOAA GSOD	`preprocess_weather.py`	Rule-based alert extraction (PRCP > 50mm, SNWD > 10cm, VISIB < 1mi); semantic description generation	Temporal windowing — 4–8 hour adaptive patches aligned to middle-mile transit windows for SRSNet input	`WEATHER_ALERTS`
Bridge Inventory	`preprocess_bridges.py`	Lat/lon → Snowflake `GEOGRAPHY` via `ST_GEOGFROMTEXT`; constraint field extraction	**No sampling** — full 600K+ inventory retained for exhaustive spatial join coverage (safety-critical)	`BRIDGE_INVENTORY_GEO`
DataCo	`preprocess_dataco.py`	Delay propensity scores; Z-score by shipping mode; late delivery risk cast	Full dataset retained for SRSNet training coverage	`CLEANED_LOGISTICS`
Logistics Ops	`preprocess_logistics.py`	Record-to-text transformation; metadata JSON construction; chunk ID assignment	Full dataset — all records vectorized for RAG retrieval completeness	`LOGISTICS_VECTORIZED`

B Retrieval & Processing Pipeline

Full documentation: docs/retrieval_processing_pipeline.md

B.1 Chunking Strategy: Semantic Record-Level

We employ a **Semantic Record-Level** chunking strategy rather than arbitrary text splitting. Structured database records from `BRONZE.LOGISTICS` are transformed into natural language sentences. Each "chunk" represents a complete logistics event (trip, maintenance record, or safety incident), preserving entity relationships.

- **Strategy:** Record-to-Text Mapping
- **Implementation:** `src/preprocessing/preprocess_logistics.py`
- **Rationale:** Logistics data is inherently transactional. A chunk is most meaningful when it represents a complete event, rather than a fixed-size text window.

Example Transformation:

Listing 5: Semantic chunking: structured data to natural language

```
# Input (structured):
Load_ID: 123, Origin: Chicago, Destination: St. Louis, Status: Late

# Output (chunk):
"Record type: load. Route: Chicago -> St. Louis, 300 miles.
 Status: Late. On-time: False."
```

Additional chunking strategies used across the pipeline:

- **Time-Series:** Adaptive patching (4–8 hour windows) for SRSNet forecasting, aligned to middle-mile transit time scales.
- **Text:** 512–1024 token segments with sliding windows for longer document inputs.
- **Geospatial:** 10-mile route segments for localized accident risk scoring.

B.2 Indexing and Retrieval Configuration

The system uses a **Dense Vector Retrieval** configuration leveraging Snowflake's native vector support:

- **Vector Type:** Snowflake `VECTOR(FLOAT, 768)` with L2 distance
- **Retrieval:** Cortex Search with ReMindRAG-guided traversal
- **Hybrid:** Dense/sparse retrieval with reranking for improved relevance
- **Metadata:** Chunks indexed alongside a JSON `METADATA` object containing `driver_id`, `truck_id`, `event_date`, and `revenue` to support hybrid filtering.
- **Table:** `HYPERLOGISTICS_DB.SILVER.LOGISTICS_VECTORIZED`

B.3 Embedding Models Used

Table 8: Embedding and LLM Models

Model	Provider	Dimensions	Purpose
`snowflake-arctic-embed-m`	Snowflake Cortex	768	Primary embedding model for RAG vector search
`snowflake-arctic`	Snowflake Cortex	—	LLM for grounded answer generation
Sentence Transformers	HuggingFace	768	Fallback for comparison/evaluation

Running the embedding model within Snowflake eliminates the need to move data to external GPU clusters, ensuring low latency and high security.

B.4 Example Retrieval Outputs

B.4.1 Query 1: "What are the recent safety concerns for driver DRV_102?"

- **Retrieved Chunk:** *"Record type: safety. Safety incident: Hard Braking involving driver DRV_102 on 2024-02-15. Details: Sudden stop on I-95 due to traffic surge."*
- **Cortex AI Response:** *"Based on the safety logs for Feb 15th, Driver DRV_102 had a hard braking incident on I-95. The driver reported a traffic surge, and no damage was recorded. Recommendation: Schedule a quick safety refresh on maintaining distance in high-traffic zones."*

B.4.2 Query 2: "Check if my route from Chicago to St. Louis is safe today."

- **Retrieved Context:**
  - *Accident Data:* 12 high-severity incidents on I-55 in the last 24 hours.
  - *Weather Alert:* "Station 724340 reports Heavy precipitation expected on 2024-02-27."
- **Cortex AI Response:** *"Warning: The standard I-55 route from Chicago to St. Louis is currently high-risk. There are 12 active incident reports and heavy rain forecast for the next 6 hours. Recommendation: Delay departure by 4 hours or reroute via I-57 to avoid the storm cell."*

B.5 Preprocessing Scripts in GitHub

All processing logic is automated via the following scripts in the repository:

- `src/preprocessing/preprocess_logistics.py` — Semantic chunking & vectorization prep
- `src/preprocessing/preprocess_accidents.py` — Risk heatmap aggregation
- `src/preprocessing/preprocess_weather.py` — Weather alert extraction
- `src/preprocessing/preprocess_bridges.py` — Spatial normalization (`GEOGRAPHY` conversion)
- `src/preprocessing/preprocess_dataco.py` — Delay propensity feature engineering
- `notebooks/srsnet_training.ipynb` — SRSNet model training notebook

C Application Integration

Full documentation: `docs/application_integration.md`

C.1 Streamlit Project Interface

The dashboard (`src/app/dashboard.py`) connects directly to Snowflake via Snowpark and provides four main tabs:

1. **Risk Heatmap** — Displays traffic incident hotspots across the US via Folium interactive maps, powered by `SILVER.RISK_HEATMAP_VIEW`.
2. **Route Comparison** — Side-by-side AI-suggested vs. original route with delay metrics and bridge clearance verification.
3. **Evidence & Reasoning** — Shows the raw grounding data (accident risk, weather alerts, bridge constraints, logistics vectors) that the AI uses to form its answers.
4. **Query Logs** — Full audit trail of every AI query, response, grounding sources, and response time.

The sidebar contains a natural language input ("Ask the AI Agent") where users can submit logistics queries. The system:

1. Retrieves relevant evidence from Silver-layer tables via `get_evidence()`.
2. Passes the evidence as context to **Snowflake Cortex** for grounded generation.
3. Displays the grounded AI response with source citations.
4. Logs the query, response, and metadata to `GOLD.QUERY_LOGS`.

C.2 Evidence Display and Grounded Answer Generation

When a user submits a query, the `get_evidence()` function retrieves supporting data from three Silver-layer tables:

Table 9: Evidence Sources for Grounded AI Responses

Source Table	Data Type	Example Evidence
`RISK_HEATMAP_VIEW`	Accident risk aggregations	"Miami, FL: 2,340 incidents, avg severity 2.8"
`BRIDGE_INVENTORY_GEO`	Bridge compliance data	"I-95 Bridge: 14.2ft clearance, 40 ton limit, Fair"
`LOGISTICS_VECTORIZED`	Operational text chunks	"Trip T-5521: Chicago → St. Louis, 297 mi, on-time"

This evidence is injected into the Cortex AI prompt as structured context, ensuring that the AI's answer is **grounded in real data** rather than hallucinated. The "Evidence & Reasoning" tab (Tab 3) allows dispatchers to inspect the raw sources behind any recommendation.

Example Grounded Response:

> **User Query:** "What are the riskiest routes near Chicago?"
>
> **AI Response:** "Based on accident data, the Chicago metropolitan area has over 15,200 recorded traffic incidents with an average severity of 2.4. The Chicago to St. Louis corridor (I-55) shows the highest concentration of delays, with 3 late deliveries in the past month. Recommendation: Consider routing via I-65 through Indianapolis for time-sensitive shipments."
>
> *Grounded on: SILVER.RISK_HEATMAP_VIEW, SILVER.LOGISTICS_VECTORIZED*

C.3 Query Logging and Evaluation Outputs

Every query submitted through the dashboard is automatically logged to `GOLD.QUERY_LOGS`:

Table 10: Query Log Schema

Column	Description
`QUERY_ID`	Unique identifier (UUID)
`USER_ID`	Snowflake username of the querying user
`QUERY_TEXT`	Raw natural language question
`RESPONSE_TEXT`	Full AI-generated response
`GROUNDING_SOURCES`	JSON array of Silver tables used as evidence
`EXECUTION_TIME_MS`	End-to-end response time in milliseconds
`IS_GROUNDED`	Boolean — TRUE if evidence was found and used
`CREATED_AT`	Timestamp of the query

The "Query Logs" tab provides three summary metrics: **Total Queries**, **Grounded %**, and **Avg Response Time**. These metrics support the evaluation framework defined in the Phase 1 proposal.

C.4 Screenshots and Deployment

- **Local Deployment:** `streamlit run src/app/dashboard.py` → `http://localhost:8501`
- **Production Target:** Streamlit in Snowflake at `https://<account>.snowflakecomputing.com/streamlit/apps/HY`
- **Technology Stack:** Streamlit + Folium (maps), Snowpark (backend), Cortex (AI engine), S3 → Snowpipe → Medallion (data pipeline)

D Snowflake Data Pipeline & Schema

Full documentation: `docs/snowflake_data_pipeline.md`

D.1 Snowflake Schema (Tables, Stages, Views)

The database HYPERLOGISTICS_DB follows a three-layer Medallion Architecture:

Listing 6: Medallion Architecture layout

```
HYPERLOGISTICS_DB
  +-- BRONZE (Raw Data)          -- Untouched data from S3
  +-- SILVER (Cleaned/Enriched)  -- Preprocessed, analytics-ready
  +-- GOLD   (Business Logic)    -- AI outputs, query logs, reports
```

Table 11: Bronze Layer Tables

Table	Source Dataset	Records	Purpose
TRAFFIC_INCIDENTS	US Accidents	~3.5M	GPS-tagged incidents for risk heatmaps
BRIDGE_INVENTORY	National Bridge Inventory	624K	Bridge clearance/load limits for safety veto
RAW_LOGISTICS	DataCo Supply Chain	180K	Historical delay patterns for SRSNet
LOGISTICS	Logistics Operations DB	2.7M	Trucking ops for RAG knowledge base
NOAA_GSOD (ext.)	NOAA GSOD	Millions	Weather disruption signals

Table 12: Silver Layer Tables

Table/View	Source	Purpose
RISK_HEATMAP_VIEW	TRAFFIC_INCIDENTS	Aggregated incident counts by State/City/Hour
BRIDGE_INVENTORY_GEO	BRIDGE_INVENTORY	Bridge data with GEOGRAPHY type for spatial joins
CLEANED_LOGISTICS	RAW_LOGISTICS	Delay propensity scores for SRSNet training
LOGISTICS_VECTORIZED	LOGISTICS	Text chunks + metadata for RAG retrieval
WEATHER_ALERTS	NOAA_GSOD	Semantic weather alerts (rain, snow, low visibility)

Table 13: Gold Layer Tables

Table	Purpose
VECTOR_EMBEDDINGS	Cortex-generated 768-dim vectors for semantic search
DISRUPTION_REPORTS	AI-generated disruption impact reports
QUERY_LOGS	Full audit trail of every AI query

Table 14: Stages & Pipes (Automated S3 Ingestion)

Stage	S3 Path	Target Table	Pipe
ACCIDENTS_STAGE	s3://bucket/accidents/	TRAFFIC_INCIDENTS	ACCIDENTS_PIPE
BRIDGES_STAGE	s3://bucket/bridges/	BRIDGE_INVENTORY	BRIDGES_PIPE
DATACO_STAGE	s3://bucket/dataco/	RAW_LOGISTICS	DATACO_PIPE
LOGISTICS_STAGE	s3://bucket/logistics/	LOGISTICS	LOGISTICS_PIPE
NOAA_S3_STAGE	s3://noaa-gsod-pds/	NOAA_GSOD (ext.)	N/A

D.2 Reproducible Ingestion Scripts (SQL / Snowpark / Python)

Scripts are run in the following order:

1. `src/sql/00_create_database.sql` — Creates database, 3 schemas, and all Bronze/Silver/Gold tables
2. `src/sql/01_setup_noaa.sql` — Creates external table pointing to public NOAA S3 bucket
3. `src/sql/02_setup_s3_automation.sql` — Creates S3 stages and Snowpipe for automated ingestion
4. `upload_to_s3.py` — Uploads local CSV files to S3 (Snowpipe auto-loads)
5. `src/preprocessing/preprocess_*.py` — Transforms Bronze → Silver via Snowpark Python

D.3 Example Queries Demonstrating Warehouse Connectivity

Listing 7: Query 1: Top accident hotspots along freight corridors

```sql
SELECT STATE, CITY, INCIDENT_COUNT, AVG_SEVERITY
FROM HYPERLOGISTICS_DB.SILVER.RISK_HEATMAP_VIEW
ORDER BY INCIDENT_COUNT DESC LIMIT 10;
```

Listing 8: Query 2: Bridge clearance check for Illinois routes

```sql
SELECT FEATURES_DESC_006A AS BRIDGE_NAME,
       VERT_CLR_OVER_MT_053 AS VERTICAL_CLEARANCE_MT,
       OPERATING_RATING_064 AS LOAD_RATING, BRIDGE_CONDITION
FROM HYPERLOGISTICS_DB.SILVER.BRIDGE_INVENTORY_GEO
WHERE VERT_CLR_OVER_MT_053 < 4.5 AND STATE_CODE_001 = 17
ORDER BY VERT_CLR_OVER_MT_053 ASC;
```

Listing 9: Query 3: Delay analysis by shipping mode (Silver layer)

```sql
SELECT SHIPPING_MODE,
       AVG(DELAY_DAYS) AS avg_delay,
       AVG(DELAY_PROPENSITY) AS avg_propensity
FROM HYPERLOGISTICS_DB.SILVER.CLEANED_LOGISTICS
GROUP BY SHIPPING_MODE
ORDER BY avg_delay DESC;
```

Listing 10: Query 4: AI-powered query via Snowflake Cortex

```sql
SELECT SNOWFLAKE.CORTEX.COMPLETE('snowflake-arctic',
    'What are the main causes of supply chain delays
     in the Midwest?');
```

Listing 11: Query 5: Audit trail of AI interactions

```sql
SELECT QUERY_TEXT, RESPONSE_TEXT, GROUNDING_SOURCES,
       EXECUTION_TIME_MS, IS_GROUNDED, CREATED_AT
FROM HYPERLOGISTICS_DB.GOLD.QUERY_LOGS
ORDER BY CREATED_AT DESC LIMIT 10;
```

D.4 How Snowflake Integrates with the Application

Table 15: Application Integration Points

Integration	**Technology**	**Purpose**
Data Ingestion	Snowpipe + S3 Events	Auto-loads CSV files from S3 into Bronze tables on arrival
Data Processing	Snowpark Python API	Preprocessing scripts transform data and write to Silver tables
AI Generation	Cortex (`COMPLETE`)	Dashboard queries the LLM for grounded answers
Vector Search	Cortex (`EMBED_TEXT_768`)	768-dim embeddings for semantic retrieval in RAG
Dashboard	Streamlit + Snowpark	Real-time queries to Silver/Gold for visualizations and metrics
Observability	`GOLD.QUERY_LOGS`	Every AI interaction logged with timing and grounding metadata

Snowflake serves as a **unified platform**: data storage, ML inference (via Cortex), and application hosting (via Streamlit in Snowflake) — eliminating the need for external services or data movement.

E Reproducibility Plan

Full documentation: `docs/reproducibility_plan.md` *and* `reproducibility/README_repoductibility.md`

E.1 Environment Configuration

Requirements.txt (pinned dependencies):

Listing 12: Key dependencies from requirements.txt

```
snowflake-snowpark-python>=1.0.0
streamlit>=1.0.0
torch>=2.0.0
transformers>=4.0.0
sentence-transformers>=2.0.0
pandas>=1.5.0
numpy>=1.24.0
boto3>=1.28.0
folium>=0.20.0
streamlit-folium>=0.26.0
python-dotenv>=1.0.0
```

Setup Commands:

Listing 13: Environment setup

```
git clone https://github.com/mosomo82/
    CS5542_SmartSC_Optimization_System.git
cd CS5542_SmartSC_Optimization_System
python -m venv venv
.\venv\Scripts\activate          # Windows
pip install -r requirements.txt
cp .env.template .env            # Fill in credentials
```

External Service Requirements:

- **Snowflake** — Enterprise Edition on AWS (Cortex AI, Snowpipe)
- **AWS S3** — Standard tier for data staging
- **Python** — 3.10 or 3.11 (Snowpark-compatible)

Environment Variables: Configured in `.env` (template at `.env.template`) with Snowflake credentials (`SNOWFLAKE_ACCOUNT`, `SNOWFLAKE_USER`, `SNOWFLAKE_PASSWORD`, `SNOWFLAKE_ROLE`, `SNOWFLAKE_WAREHOUSE`, `SNOWFLAKE_DATABASE`, `SNOWFLAKE_SCHEMA`), AWS keys, S3 bucket name, and region.

E.2 Model Versions and Dataset Versioning

Table 16: AI Model Versions (Snowflake-Managed — No Local GPU Required)

Model	Provider	Version	Purpose
`snowflake-arctic`	Snowflake Cortex	Latest (managed)	LLM for grounded answers
`snowflake-arctic-embed-m`	Snowflake Cortex	Latest (managed)	768-dim embeddings for RAG

Table 17: Dataset Versions

Dataset	Version	Records	Source
US Accidents	2016–2023 (v14)	∼7.7M	Kaggle
DataCo Supply Chain	v1 (2020)	∼180K	Kaggle
National Bridge Inventory	NBI 2024	∼600K	BTS/FHWA
NOAA GSOD	Continuous (public)	Millions	AWS Open Data
Logistics Operations DB	v1	∼550K	Kaggle

Data Versioning Strategy:

- **Bronze tables** contain raw, immutable snapshots of source CSVs.
- **Silver tables** are deterministically regenerated from Bronze by the preprocessing scripts.
- **Gold tables** contain runtime data (query logs) and are not expected to be identical across reproductions.
- SRSNet and ReMindRAG implementations are pinned to their NeurIPS 2025 paper versions.

E.3 Random Seeds and Configuration Files

The preprocessing pipeline is **deterministic by design** — no random sampling or stochastic operations. All transformations use aggregations (`GROUP BY`, `AVG`, `COUNT`), column derivations (`withColumn`), and window functions (`Window.partitionBy`).

For SRSNet model training (`notebooks/srsnet_training.ipynb`):

Listing 14: Random seed configuration

```
import torch, numpy as np, random
SEED = 42  # Defined in src/config/config.yaml
random.seed(SEED); np.random.seed(SEED)
torch.manual_seed(SEED); torch.cuda.manual_seed_all(SEED)
torch.backends.cudnn.deterministic = True
```

Configuration Files:

Table 18: Configuration Files

File	Purpose
`src/config/config.yaml`	Global seed (42), SRSNet hyperparams, ReMindRAG params, dataset paths
`.env` / `.env.template`	Snowflake and AWS credentials
`requirements.txt`	Pinned Python package versions
`.gitignore`	Ensures credentials and data files are not committed
`src/sql/00_create_database.sql`	Database schema definition (DDL)
`src/sql/01_setup_noaa.sql`	NOAA external table configuration
`src/sql/02_setup_s3_automation.sql`	S3 stages, pipes, and integration setup

E.4 Run Instructions

Estimated Total Time: ∼2 hours

1. **Environment Setup (∼10 min):** Clone repo, create venv, install `requirements.txt`, configure `.env` from template.
2. **Snowflake Setup (∼20 min):** Run SQL scripts *in order* in Snowflake Worksheets:
   a. `src/sql/00_create_database.sql` — Database, schemas, tables

   b. `src/sql/01_setup_noaa.sql` — NOAA external table
   c. `src/sql/02_setup_s3_automation.sql` — S3 stages and Snowpipe

3. **Data Loading (∼30 min):** Download datasets from source links (Table 16), place CSVs in `data/`, then run `python upload_to_s3.py`.

4. **Preprocessing (∼15 min):**

```
python src/preprocessing/preprocess_accidents.py
python src/preprocessing/preprocess_bridges.py
python src/preprocessing/preprocess_dataco.py
python src/preprocessing/preprocess_logistics.py
python src/preprocessing/preprocess_weather.py
```

5. **Launch Dashboard (∼5 min):** `streamlit run src/app/dashboard.py` → opens at `http://localhost:8501`

6. **Verify:**

```sql
-- Check Bronze layer
SELECT COUNT(*) FROM HYPERLOGISTICS_DB.BRONZE.TRAFFIC_INCIDENTS;
-- Check Silver layer
SELECT COUNT(*) FROM HYPERLOGISTICS_DB.SILVER.RISK_HEATMAP_VIEW;
-- Test Cortex AI
SELECT SNOWFLAKE.CORTEX.COMPLETE('snowflake-arctic',
    'What causes supply chain delays?');
-- Check query logs
SELECT * FROM HYPERLOGISTICS_DB.GOLD.QUERY_LOGS
ORDER BY CREATED_AT DESC LIMIT 5;
```

Table 19: Expected Reproduction Checkpoints

Checkpoint	Expected Result
Database exists	`SHOW DATABASES LIKE 'HYPERLOGISTICS%';` returns 1 row
Bronze tables populated	4+ tables with data (3.5M accidents, 180K logistics, etc.)
Silver tables created	5 tables/views with preprocessed data
Dashboard loads	Streamlit app at `localhost:8501` with 4 tabs
AI responds	Sidebar query returns a grounded response from Cortex
Query logs recorded	`GOLD.QUERY_LOGS` shows logged queries

References

[1] Wu, X., Qiu, X., et al. (2025). *Enhancing Time Series Forecasting through Selective Representation Spaces: A Patch Perspective*. arXiv preprint arXiv:2510.14510. https://arxiv.org/abs/2510.14510

[2] Hu, Y., Zhu, J., et al. (2025). *ReMindRAG: Low-Cost LLM-Guided Knowledge Graph Traversal for Efficient RAG*. arXiv preprint arXiv:2510.13193. https://arxiv.org/abs/2510.13193

[3] Maggiar, A., Dicker, L., & Mahoney, M. W. (2024). *Consensus Planning with Primal, Dual, and Proximal Agents*. Amazon Science. https://www.amazon.science/publications/consensus-planning-with-primal-dual-and-proximal-agents

[4] Snowflake Inc. (2025). *Snowpark Developer Guide: Python*. https://docs.snowflake.com/en/developer-guide/snowpark/python/index

[5] Constante, F. (2019). *DataCo SMART SUPPLY CHAIN FOR BIG DATA ANALYSIS* [Dataset]. Mendeley Data. https://www.kaggle.com/datasets/shashwatwork/dataco-smart-supply-chain-for-big-data-analysis

[6] Moosavi, S., Samavatian, M. H., Parthasarathy, S., & Ramnath, R. (2019). *A Countrywide Traffic Accident Dataset*. In Proceedings of the 27th ACM SIGSPATIAL. https://www.kaggle.com/datasets/sobhanmoosavi/us-accidents

[7] Bureau of Transportation Statistics. (2025). *National Tunnel & Bridge Inventory* [Dataset]. U.S. Department of Transportation. https://geodata.bts.gov/datasets/national-bridge-inventory/

[8] National Oceanic and Atmospheric Administration. (2025). *Global Surface Summary of the Day (GSOD)* [Dataset]. AWS Open Data Registry. https://registry.opendata.aws/noaa-gsod/

[9] Yogesh (2023). *Logistics Operations Database* [Dataset]. Kaggle. https://www.kaggle.com/datasets/yogape/logistics-operations-database